General Network Equipment Audit: Granting Read-Only Access to SNMP Clients

In a **network equipment audit**, it is essential to ensure that **SNMP (Simple Network Management Protocol) access** is correctly configured. One best practice is to **grant read-only access** to prevent unauthorized modifications while still allowing network monitoring.

Understanding the Configuration

The following configuration snippet defines **SNMP community-based authentication** with **read-only access**:

```
snmp {
    community public {
        authorization read-only;
    }
}
```

Explanation:

- **`snmp`** → Defines SNMP settings on the network device.
- **`community public`** → Creates an SNMP **community string** named `"public"` (used for authentication).
- **`authorization read-only;`** → Restricts access to **read-only mode**, meaning:
  - Clients can **retrieve information** using SNMP commands (e.g., `snmpget`, `snmpwalk`).
  - Clients **cannot modify** configurations or change device settings.

How This Works

- The system will **respond to SNMP requests** (`Get`, `GetNext`, `GetBulk`) that include the `"public"` community string.
- The response will contain **read-only data**, such as:
  - **Interface status** (up/down)
  - **CPU and memory usage**
  - **Traffic statistics**
  - **Device uptime**
- **Write operations** (e.g., `Set` requests) will be **denied** to prevent unauthorized changes.

Types of SNMP Requests Allowed

The configuration allows the following SNMP request types:

1. **`Get`** – Retrieves a specific piece of information.
    - Example: Querying **system uptime**

```
snmpget -v2c -c public 192.168.1.1 sysUpTime.0
```

2. **`GetNext`** – Retrieves the next object in a table.
    - Example: Getting **interface details** from a switch.

```
snmpgetnext -v2c -c public 192.168.1.1 ifDescr
```

3. **`GetBulk`** – Retrieves multiple values at once (efficient for large tables).
    - Example: Retrieving **all active network interfaces**

```
snmpbulkget -v2c -c public 192.168.1.1 ifTable
```

Security Considerations

While granting read-only SNMP access is **safer than read-write access**, there are still **security risks** with using a **default community string (`public`)**:

✅ Best Practices for Secure SNMP Configuration

✓ Change the default community string

- Instead of `"public"`, use a **unique, complex string** (e.g., `"NetMonitor123!"`).

```
community NetMonitor123 {
    authorization read-only;
}
```

✓ Use SNMPv3 instead of SNMPv1/v2c

- SNMPv3 supports **encryption and authentication** to prevent unauthorized access.

✓ Restrict SNMP access to specific IPs

- Example: Allow SNMP queries only from **trusted monitoring servers**.

```
community NetMonitor123 {
    authorization read-only;
    clients 192.168.1.100;
```

```
}
```

✓ Disable SNMP if not needed

- If SNMP is **not required**, disable it to **reduce attack surfaces**.

Conclusion

- The configuration allows **read-only SNMP access** for monitoring network devices.
- It enables safe **SNMP queries** but prevents **unauthorized modifications**.
- **Security improvements** (changing community strings, using SNMPv3, and IP restrictions) should be implemented to **prevent SNMP-based attacks**.

General Network Equipment Audit: Granting Read-Write SNMP Access to pingMIB and jnxPingMIB

This configuration allows **read-write SNMP access** for all clients but **only** to specific **MIBs (Management Information Bases):**

- **pingMIB** → Used for monitoring and testing network connectivity via SNMP.
- **jnxPingMIB** → A Juniper-specific MIB for handling ping-related operations on Juniper devices.

The system will respond to **SNMP Get, GetNext, GetBulk, and Set requests** when they:
✓ Use the **community string** `"private"` for authentication.
✓ Access an **OID (Object Identifier)** within the **pingMIB or jnxPingMIB** hierarchy.

Understanding the Configuration

```
plaintext
CopyEdit
snmp {
    view ping-mib-view {
        oid pingMIB include;
        oid jnxPingMIB include;
        community private {
            authorization read-write;
            view ping-mib-view;
        }
    }
}
```

Breakdown of the Configuration

1. **`snmp {}`** → Defines SNMP settings on the device.
2. **`view ping-mib-view {}`** → Creates an SNMP **view** called `"ping-mib-view"`, which allows access only to specific MIBs:
   - **`oid pingMIB include;`** → Grants access to the standard **pingMIB**.
   - **`oid jnxPingMIB include;`** → Grants access to the **Juniper-specific jnxPingMIB**.
3. **`community private {}`** → Defines the **SNMPv2c community string** (`"private"`), which acts like a password for access.
4. **`authorization read-write;`** → Allows both **reading (Get, GetNext, GetBulk)** and **writing (Set requests)** for SNMP objects within **pingMIB and jnxPingMIB**.
5. **`view ping-mib-view;`** → Restricts access to **only** the OIDs in `pingMIB` and `jnxPingMIB`.

How This Works in Practice

With this configuration, SNMP clients can:
✓ **Monitor network connectivity** using `pingMIB`.
✓ **Send and modify ping requests** using `jnxPingMIB` on Juniper devices.

However, clients **cannot**:
✗ Access or modify **other SNMP data** (e.g., interface configurations, system settings).
✗ Execute SNMP `Set` requests **outside** of `pingMIB` and `jnxPingMIB`.

Types of SNMP Requests Allowed

1 Get Request (Retrieve Ping Statistics)

Clients can retrieve **ping statistics** using `pingMIB`:

```
snmpget -v2c -c private 192.168.1.1 pingMIB.1
```

➞ Returns the number of pings sent from the device.

---

2 GetBulk Request (Retrieve Multiple Ping Results)

Retrieve **multiple ping results** at once:

```
snmpbulkget -v2c -c private 192.168.1.1 pingMIB
```

➞ Returns a list of ping-related values.

---

3 Set Request (Initiate a Ping Test Remotely)

A client can remotely **initiate a ping test** using `jnxPingMIB`:

```
snmpset -v2c -c private 192.168.1.1 jnxPingMIB.1 s "8.8.8.8"
```

➞ This command tells the device to **ping 8.8.8.8** and store the result.

Security Considerations 🔒

● **Risk: SNMPv2c is Insecure**

- The **community string ("private") is sent in plain text** over the network.
- An attacker **could sniff SNMP traffic**, capture credentials, and modify ping settings.
  ✅ **Solution: Use SNMPv3 for authentication & encryption**

● **Risk: Read-Write Access Can Be Abused**

- A malicious user **could send frequent pings** to an external host, leading to **DDoS-like behavior**.
  ✅ **Solution: Restrict access to trusted IPs**

```
community private {
    authorization read-write;
    view ping-mib-view;
    clients 192.168.1.100;
}
```

➞ Now, **only 192.168.1.100** can send SNMP requests.

● **Risk: Community String `"private"` is Weak**

- `"private"` is a well-known default string.
  ✅ **Solution: Use a stronger, unique community string:**

```
community NetSecRW123! {
    authorization read-write;
    view ping-mib-view;
}
```

Conclusion

✓ This configuration **grants read-write SNMP access** only for **pingMIB and jnxPingMIB**, ensuring controlled access.
✓ It allows remote **monitoring and execution of ping tests** without exposing other SNMP data.
✓ **Security enhancements** (SNMPv3, IP restrictions, and stronger community strings) should be applied to prevent misuse.

General Network Equipment Audit: Restricting SNMP Access Based on IP Address

This configuration **controls SNMP access** by allowing **read-only access** to specific IPv4 clients while **denying** access to certain IPv6 clients. This ensures that only **trusted devices** can retrieve SNMP data.

Understanding the Configuration

```
snmp {
    community field-service {
        authorization read-only;
        clients {
            default restrict;
            1.2.3.4/24; # Allow access by all clients in 1.2.3.4/24 except
            fe80::1:2:3:4/64 restrict; # fe80::1:2:3:4/64 is not allowed
        }
    }
}
```

Breakdown of the Configuration

1. **`snmp {}`** → Defines SNMP settings on the device.
2. **`community field-service {}`** → Creates an SNMP **community string** (`"field-service"`), which acts like a password for authentication.
3. **`authorization read-only;`** → Grants **read-only access**, meaning:
   - Clients can retrieve **network statistics, device information, and performance data**.
   - Clients **cannot modify** configurations or make changes to the device.
4. **`clients {}`** → Specifies **which IPs can access SNMP data**:
   - **`default restrict;`** → **Denies SNMP access to all clients** by default unless explicitly allowed.
   - **`1.2.3.4/24;`** → **Allows access** to all devices in the **IPv4 subnet `1.2.3.4/24`**.
   - **`fe80::1:2:3:4/64 restrict;`** → **Denies access** to any device in the **IPv6 subnet `fe80::1:2:3:4/64`**.

How This Works in Practice

- Devices with **IPv4 addresses in `1.2.3.4/24`** ✓ **CAN access SNMP data** using the `"field-service"` community string.
- Devices with **IPv6 addresses in `fe80::1:2:3:4/64`** ✗ **CANNOT access SNMP data**.

- All **other devices** (not in `1.2.3.4/24`) are **automatically denied** due to `default restrict;`.

Example SNMP Commands Based on the Configuration

✅ Allowed SNMP Query (from `1.2.3.10`)

A system **within** the allowed range can retrieve system uptime:

```
snmpget -v2c -c field-service 1.2.3.1 sysUpTime.0
```

➡ This request succeeds ✅ because **1.2.3.10 is in `1.2.3.4/24`**.

❌ Blocked SNMP Query (from `fe80::1:2:3:4:abcd`)

A system **within** the restricted IPv6 range tries to access SNMP data:

```
snmpget -v2c -c field-service [fe80::1:2:3:4:abcd] sysUpTime.0
```

➡ This request **fails** ❌ because **fe80::1:2:3:4/64 is explicitly denied**.

Security Considerations 🚨

🔴 Risk: SNMPv2c is Insecure

- The **community string (`field-service`) is sent in plaintext** over the network.
  ✅ **Solution: Upgrade to SNMPv3 for encryption & authentication.**

🔴 Risk: "field-service" Community String is Guessable

- Attackers could try **brute-force guessing** the SNMP community string.
  ✅ **Solution: Use a stronger, unique SNMP community string:**

```
community NetSecReadOnly123! {
    authorization read-only;
}
```

● **Risk: Default Allow vs. Default Deny**

- This configuration is **secure because it follows "default deny"** (`default restrict;`).
- If **default allow** were used, anyone could query SNMP data.
  ✅ **Solution: Always use `default restrict;` and allow only trusted IPs.**

Conclusion

✓ **Only IPv4 clients in `1.2.3.4/24`** can retrieve SNMP data.
✓ **IPv6 clients in `fe80::1:2:3:4/64` are explicitly blocked**.
✓ **All other clients are denied by default** for enhanced security.
✓ **Use SNMPv3 for secure authentication & encryption**.